GH-300

Prompt Engineering

for Senior Developers

Prompts are specifications. Patterns matter. Context engineering wins.

8 elements | Few-Shot | CoT | Step-back | Decomposition | Context Eng

Sandeep Soni

Deck 1 · GH-300 Certified GitHub Copilot Expert Training

What is Prompt Engineering?

The craft of designing effective instructions that guide an AI model toward the output you actually want — not the output that sounds right.

Crafting inputs

Design the prompt deliberately — the same way you design an API contract.

Vital for quality

Optimises responses; ambiguous prompts cause unpredictable results.

Intent · context · clarity

Goes beyond syntax. Structured prompts with examples improve consistency.

Addresses risk

Mitigates hallucination, bias, and unsafe outputs — when used as a discipline.

The paradigm shift — from writing code to directing AI

Think of yourself as an architect with a very capable but literal-minded construction crew. You write the spec; the crew implements it.

	Traditional development	AI-assisted development
Workflow	Requirement → Design → Code	Requirement → Prompt → Review → Refine
Implementation	You implement every detail	You specify intent; AI generates
Time spent	~80% typing, 20% thinking	Context-engineering + reviewing dominate
Key skill	Language syntax mastery	Clear specification + critical evaluation

Prompts ARE specifications — same rigor, different syntax

Specification principles

Ambiguity leads to unexpected results.

Completeness prevents gaps.

Precision ensures correctness.

Same principles you already apply to API contracts and technical design docs. A prompt is a specification for an AI system.

spec-as-prompt

```
// Precise prompt — email validation

Write a C# function that validates email addresses.

Requirements:
  1. Return bool indicating validity.
  2. Check for @ symbol and valid domain.
  3. Null / empty → return false.
  4. Use regex for pattern matching.
  5. Include XML documentation.

// Why it works: language, return type,
// validation rules, edge cases,
// implementation approach, documentation.
```

The eight elements of an effective prompt

1 Role

Who should the AI act as? Sets its mental frame.

2 Task

Action-oriented and clear; vague terms add noise.

3 Audience

Shapes tone, complexity, and style for the reader.

4 Context

The invisible contract — surrounding code, patterns, libs.

5 Constraints

Rules that limit or guide: language, length, must-haves.

6 Output format

Structure for consistency and easy evaluation.

7 Examples

Positive + negative samples improve pattern recognition.

8 Tool / agent scope

NEW — what tools the AI may use, what it may NOT touch.

Element 8 — "Tool / agent scope" — becomes load-bearing when prompts run in agent mode. Decide what the AI can read, edit, run BEFORE you let it loose.

Role & Audience — setting the frame and the reader

Role patterns

- Generic: "Help me with this code."
- Role-based: "As a senior .NET architect, review this code."
- "As a DBA optimising for SQL Server 2022, write the query."
- "As a debugging expert, analyse this stack trace."

Audience + Role combined

- "As a tech lead, explain microservices to a junior developer who knows monoliths."
- "As a security expert, review this auth code and explain vulnerabilities to a developer unfamiliar with OWASP."
- "As a performance engineer, explain LINQ optimisation to someone who knows SQL but not EF internals."

Why the pair matters

Role alone tells the AI HOW to think. Audience alone tells it WHO to think for. Together they collapse the explanation to the right altitude — and steer Copilot away from generic, hedged answers toward decisions you can actually use.

Task clarity — every word adds signal or adds noise

Level 1 — Vague	*"Write good code"*
Level 2 — Better	*"Write maintainable code"*
Level 3 — Good	*"Write code with clear names and single-responsibility functions"*
Level 4 — Precise	*"Write Python, Use SOLID Principles, methods < 20 lines, snake_case, docstrings for public members"*

Quantify when possible

"Make it fast" → **"Optimise for O(n) time complexity"**

"Add error handling" → **"try/catch, log to ILogger, return Result<T>"**

"Write tests" → **"xUnit: 3 happy + 2 edge cases, parameterised"**

"Keep it simple" → **"No external deps, max 50 lines"**

Context · Constraints · Output · Examples — the other four levers

Context

The invisible contract. Without it, the AI makes assumptions. "Used in a registration system. Single string input. Surrounding code uses Result<T>."

Constraints

Rules that limit or guide. "Python 3.12 only. Built-in libraries. Simple and readable."

Output format

Structure for consistency and easy evaluation. "Return: (1) the function, (2) a 2-line explanation, (3) 3 test cases."

Examples

Positive AND negative samples improve pattern recognition. "Valid: user@example.com. Invalid: @gmail.com, usergmail.com."

Positive AND negative examples beat positive-only examples almost every time.

Complete example — all eight elements assembled

full-prompt

```
# Email validator — full prompt

Role:         Senior Python engineer with production experience.
Task:         Validate whether a string is a valid email address.
Audience:     Junior devs who will reuse this in a backend app.
Context:      User-registration system. Single string input.
Constraints:  Python 3.12. Regex-based. No external libraries. Simple and readable.
Output:       (1) function, (2) 2-line explanation, (3) 3 test cases.
Examples:     Valid: user@example.com, john.doe@gmail.com.
              Invalid: @gmail.com, user@com, usergmail.com.
Tool scope:   AI may EDIT a new file under /validators. AI may NOT touch /tests.
              On completion, hand off to the test-writer agent.
```

Six prompt patterns — match the technique to the task

Zero-Shot vs Few-Shot

Standard tasks vs custom conventions with examples.

Chain of Thought

Step-by-step reasoning for complex logic.

Step-back Prompting

Broader context first, then specifics. (Now structural — see Plan mode.)

Self-Verification

Generate, verify, improve. Self-checking.

Decomposition

Break complex tasks into prompt chains.

Iterative Refinement

Generate → Critique → Improve cycle.

Pattern 1 — Zero-Shot vs Few-Shot

Zero-Shot

No examples. Rely entirely on training. Best for standard tasks, common patterns, well-documented frameworks.

Few-Shot

Provide examples that demonstrate the pattern. Best for custom naming conventions, unusual output formats, company-specific code style.

few-shot prompt

```
// Few-Shot example — generate the next instance from two examples

Example 1:  async def get_user_by_id(self, user_id: int) -> Result[UserDTO]:
              user = await self.repo.find_by_id(user_id)
              return Result.not_found(...) if user is None else Result.success(UserDTO.from_entity(user))

Example 2:  async def delete_product(self, product_id: int) -> Result[bool]:
              ...same pattern...

Now generate: async def update_customer_address(self, customer_id, address)
```

Pattern 2 — Chain of Thought (CoT)

Give the model the steps to reason through: what to trace, which values to show, what to check. Costs more tokens; pays off on multi-step problems.

without CoT

```
def calc_total(order):
    sub = sum(i.price * i.qty for i in order.items)
    discount = 0.1 if order.is_premium else 0
    tax = sub * 0.08
    return sub - discount + tax

// Without CoT
"Why does this return wrong results?"

→ AI spots one issue, misses the other.
```

with CoT

```
// With guided CoT
"Trace calc_total step by step
 for a premium member with a
 $100 subtotal. Show each
 intermediate value. Then list
 every bug with a fix."

What the trace exposes
 – discount is a flat $0.10, not 10%
 – tax is charged before the discount
→ Both bugs found, each with a fix.
```

Reasoning models: set the effort, specify the checks

Models with a reasoning-effort setting think before they answer. "Think step by step" adds tokens, not accuracy; the prompt's job is the goal, the rules and the checks.

non-reasoning model

```
// e.g. GPT-4.1, fast/low-cost models
// You write the reasoning steps
"Trace calc_total step by step
 for a premium member with a
 $100 subtotal. Show each
 intermediate value. Then list
 every bug with a fix."

Use when
 – the model has no effort setting
 – you need a trace you can review
→ Guided CoT from Pattern 2.
```

reasoning model

```
// e.g. o-series, GPT-5, Claude thinking
// effort: medium. State goal + rules
"Find every bug in calc_total.
 Rules: premium = 10% off the
 subtotal; tax applies after the
 discount. Return each bug,
 its impact, and a fix."

Set effort by task
 – high: multi-file, multi-step bugs
 – low: renames, simple edits
→ Drop "think step by step".
```

Pattern 3 — Step-back prompting (now product-shaped in Plan mode)

Before solving X, first consider Y, then apply that understanding to X. Prevents narrow solutions that miss the bigger picture.

step-back prompt

```
// With Step-back  – database choice example

Before recommending a database, first analyse these factors:
  1. Data characteristics:
     - Product catalog: 50k products, complex nested attributes.
     - Orders: 10k/day, strict consistency, complex joins for reporting.
     - User sessions: 100k concurrent, simple key-value, 24h TTL.
     - Analytics events: 5M/day, append-only, queried by date range.
  2. Team expertise: strong PostgreSQL background; limited NoSQL.
  3. Constraints: Azure cloud; must integrate with existing Django app.
```

Connection to product: Copilot Plan mode is step-back prompting institutionalised — review the plan before any code is written.

Pattern 4 — Self-Verification (four sub-patterns)

Generate then verify

Write a function; trace through the test cases; if any fail, fix and re-show.

Multiple approaches

Provide 3 different solutions with pros, cons, use cases. Then recommend best fit.

Devil's advocate

Explain strengths first. Then critically review for problems. Suggest improvements.

Bug catching

Generate code; verify by tracing test cases and checking complexity claims.

Pattern: Generate → Critique → Improve. Asking the model to self-check beats accepting the first answer almost every time.

Patterns 5 + 6 — Decomposition and Iterative Refinement

Decomposition

Break a complex task into a chain of focused prompts. Single mega-prompts produce shallow output; chained prompts produce coherent systems.

Example chain

① Design — models, schemas, ERD.

② Business logic — services, methods.

③ API layer — endpoints, validation.

④ Integration — email, async, retry.

⑤ Testing — unit + integration + edges.

Iterative Refinement

Do not accept the first output. Critique → improve. Mimics the code-review-and-iteration cycle senior devs already do.

Three-step pattern

Step 1 — Generate. "Write a Python YAML config parser with env-var substitution."

Step 2 — Critique. "Review for error handling, security, large-file perf, edge cases. List issues with line refs."

Step 3 — Improve. "Refactor to fix all issues. Comment each fix."

Context Engineering — the operational layer above prompts

Prompt engineering is what you say. Context engineering is everything the AI sees alongside what you say — the script, the stage, the props. In Copilot, this is now a discipline with its own primitives.

Custom Instructions

.github/copilot-instructions.md + AGENTS.md — teach Copilot HOW you write code, always.

Prompt files

.github/prompts/*.prompt.md — reusable, parameterised prompts as slash commands.

Agent Skills

.github/skills/*/SKILL.md — capabilities loaded on demand when the description matches.

Custom Agents

.github/agents/*.agent.md — scoped personas with locked tools and pinned models.

Context engineering is like being a stage director for the AI — you do not change the actor (the model); you set the scene, hand it the right script, and make sure the spotlight is on the important details.

Thank you.

Pick one repo. Write its copilot-instructions.md by Friday.

Sandeep Soni

GH-300 · Prompt Engineering · Deck 1